

Classicmodels API paths (main.py)

This notebook calls the `/customers`, `/orders`, and `/orderdetails` routes from `app/main.py` over HTTP using the `requests` package.

Prerequisite: Run the API from the repository root, for example `uvicorn app.main:app --reload --port 8000`. The default base URL is `http://127.0.0.1:8000`; override with the environment variable `API_BASE_URL` if you use another host or port.

Note: Cells that **POST**, **PUT**, or **DELETE** modify the `classicmodels` database. Run create/update/delete cells once, or re-run only after the created rows have been deleted.

```
In [1]: import os
import requests

BASE_URL = os.environ.get("API_BASE_URL", "http://127.0.0.1:8000").rstrip("/")

try:
    _health = requests.get(f"{BASE_URL}/health", timeout=5)
    _health.raise_for_status()
except requests.RequestException as exc:
    raise RuntimeError(
        f"Cannot reach API at {BASE_URL!r}. From the repo root run e.g. "
        "`uvicorn app.main:app --reload --port 8000`, then rerun this cell "
        "(or set API_BASE_URL if the server uses another host/port)."
    ) from exc

# Sample PKs from the standard classicmodels dataset.
SAMPLE_CUSTOMER_NUMBER = 103          # Atelier graphique
SAMPLE_ORDER_NUMBER     = 10100       # first order in the dataset
SAMPLE_PRODUCT_CODE     = "S18_1749" # a line item on order 10100
```

Customers

GET /customers

List/search with optional query parameters: `customerName`, `city`, `country` (equality template).

```
In [2]: resp = requests.get(f"{BASE_URL}/customers", timeout=30)
assert resp.status_code == 200, resp.text
data = resp.json()
assert "items" in data
print("count:", len(data["items"]))
data["items"][:3]
```

count: 122

```
Out[2]: [{'customerNumber': 103,
'customerName': 'Atelier graphique',
'contactLastName': 'Schmitt',
'contactFirstName': 'Carine ',
'phone': '40.32.2555',
'addressLine1': '54, rue Royale',
'addressLine2': None,
'city': 'Nantes',
'state': None,
'postalCode': '44000',
'country': 'France',
'salesRepEmployeeNumber': 1370,
'creditLimit': '21000.00'},
{'customerNumber': 112,
'customerName': 'Signal Gift Stores',
'contactLastName': 'King',
'contactFirstName': 'Jean',
'phone': '7025551838',
'addressLine1': '8489 Strong St.',
'addressLine2': None,
'city': 'Las Vegas',
'state': 'NV',
'postalCode': '83030',
'country': 'USA',
'salesRepEmployeeNumber': 1166,
'creditLimit': '71800.00'},
{'customerNumber': 114,
'customerName': 'Australian Collectors, Co.',
'contactLastName': 'Ferguson',
'contactFirstName': 'Peter',
'phone': '03 9520 4555',
'addressLine1': '636 St Kilda Road',
'addressLine2': 'Level 3',
'city': 'Melbourne',
'state': 'Victoria',
'postalCode': '3004',
'country': 'Australia',
'salesRepEmployeeNumber': 1611,
'creditLimit': '117300.00'}]
```

```
In [3]: resp = requests.get(
    f"{BASE_URL}/customers", params={"country": "France"}, timeout=30
)
assert resp.status_code == 200
items = resp.json()["items"]
assert all(r["country"] == "France" for r in items)
len(items)
```

Out[3]: 12

GET /customers/{customerNumber}

Returns **404** if the customer does not exist.

```
In [4]: resp = requests.get(f"{BASE_URL}/customers/{SAMPLE_CUSTOMER_NUMBER}", timeout=30)
assert resp.status_code == 200
resp.json()
```

```
Out[4]: {'customerNumber': 103,
'customerName': 'Atelier graphique',
'contactLastName': 'Schmitt',
'contactFirstName': 'Carine ',
'phone': '40.32.2555',
'addressLine1': '54, rue Royale',
'addressLine2': None,
'city': 'Nantes',
'state': None,
'postalCode': '44000',
'country': 'France',
'salesRepEmployeeNumber': 1370,
'creditLimit': '21000.00'}
```

```
In [5]: missing = requests.get(f"{BASE_URL}/customers/999999", timeout=30)
assert missing.status_code == 404
missing.json()
```

```
Out[5]: {'detail': "No customer with customerNumber '999999'"}
```

POST /customers

Creates a customer. Response body is the new customerNumber .

```
In [6]: payload = {
    "customerNumber": 50000,
    "customerName": "Notebook Test Customer",
    "contactLastName": "Test",
    "contactFirstName": "Notebook",
    "phone": "555-0100",
    "addressLine1": "1 Test Street",
    "city": "New York",
    "country": "USA",
}
resp = requests.post(f"{BASE_URL}/customers", json=payload, timeout=30)
assert resp.status_code == 200, resp.text
new_customer_id = resp.json()
print("created customerNumber:", new_customer_id)
new_customer_id
```

created customerNumber: 50000

```
Out[6]: '50000'
```

PUT /customers/{customerNumber}

Updates by customerNumber ; **400** if the customer does not exist.

```
In [7]: # Uses `new_customer_id` from the POST cell above – run that cell first.
update_body = {
    "customerNumber": new_customer_id,
    "customerName": "Notebook Test Customer (updated)",
    "contactLastName": "Test",
    "contactFirstName": "Notebook",
    "phone": "555-0199",
    "addressLine1": "1 Test Street",
    "city": "Boston",
    "country": "USA",
}
resp = requests.put(f"{BASE_URL}/customers/{new_customer_id}", json=update_body, timeout=30)
assert resp.status_code == 200, resp.text
assert resp.json() == {"updated": 1}
requests.get(f"{BASE_URL}/customers/{new_customer_id}", timeout=30).json()
```

```
Out[7]: {'customerNumber': 50000,
'customerName': 'Notebook Test Customer (updated)',
'contactLastName': 'Test',
'contactFirstName': 'Notebook',
'phone': '555-0199',
'addressLine1': '1 Test Street',
'addressLine2': None,
'city': 'Boston',
'state': None,
'postalCode': None,
'country': 'USA',
'salesRepEmployeeNumber': None,
'creditLimit': None}
```

DELETE /customers/{customerNumber}

Returns {"deleted": 0} or {"deleted": 1}.

```
In [8]: resp = requests.delete(f"{BASE_URL}/customers/{new_customer_id}", timeout=30)
assert resp.status_code == 200
assert resp.json()["deleted"] == 1

gone = requests.get(f"{BASE_URL}/customers/{new_customer_id}", timeout=30)
assert gone.status_code == 404
gone.json()
```

```
Out[8]: {'detail': 'No customer with customerNumber '50000'...'}
```

Orders

GET /orders

List/search with optional query parameters: status , customerNumber (equality template).

```
In [9]: resp = requests.get(f"{BASE_URL}/orders", timeout=30)
assert resp.status_code == 200, resp.text
data = resp.json()
assert "items" in data
print("count:", len(data["items"]))
data["items"][:3]
```

count: 326

```
Out[9]: [{ 'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363},
{ 'orderNumber': 10101,
  'orderDate': '2003-01-09',
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-11',
  'status': 'Shipped',
  'comments': 'Check on availability.',
  'customerNumber': 128},
{ 'orderNumber': 10102,
  'orderDate': '2003-01-10',
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-14',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 181}]
```

```
In [10]: resp = requests.get(
    f"{BASE_URL}/orders", params={"status": "Shipped"}, timeout=30
)
assert resp.status_code == 200
items = resp.json()["items"]
assert all(r["status"] == "Shipped" for r in items)
len(items)
```

Out[10]: 303

GET /orders/{orderNumber}

Returns **404** if the order does not exist.

```
In [11]: resp = requests.get(f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}", timeout=30)
assert resp.status_code == 200
resp.json()
```

```
Out[11]: { 'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363}
```

```
In [12]: missing = requests.get(f"{BASE_URL}/orders/999999", timeout=30)
assert missing.status_code == 404
missing.json()
```

```
Out[12]: {'detail': 'No order with orderNumber '999999'}
```

POST /orders

Creates an order. Response body is the new orderNumber .

```
In [13]: order_payload = {
    "orderNumber": 99999,
    "orderDate": "2024-01-15",
    "requiredDate": "2024-01-30",
    "status": "In Process",
    "customerNumber": SAMPLE_CUSTOMER_NUMBER,
}
resp = requests.post(f"{BASE_URL}/orders", json=order_payload, timeout=30)
assert resp.status_code == 200, resp.text
new_order_id = int(resp.json())
print("created orderNumber:", new_order_id)
new_order_id
```

created orderNumber: 99999

Out [13]: 99999

PUT /orders/{orderNumber}

Updates by orderNumber ; **400** if the order does not exist.

```
In [14]: # Uses `new_order_id` from the POST cell above – run that cell first.
order_update = {
    "orderNumber": new_order_id,
    "orderDate": "2024-01-15",
    "requiredDate": "2024-01-30",
    "status": "Shipped",
    "customerNumber": SAMPLE_CUSTOMER_NUMBER,
}
resp = requests.put(f"{BASE_URL}/orders/{new_order_id}", json=order_update, timeout=30)
assert resp.status_code == 200, resp.text
assert resp.json() == {"updated": 1}
requests.get(f"{BASE_URL}/orders/{new_order_id}", timeout=30).json()
```

```
Out [14]: {'orderNumber': 99999,
'orderDate': '2024-01-15',
'requiredDate': '2024-01-30',
'shippedDate': None,
'status': 'Shipped',
'comments': None,
'customerNumber': 103}
```

DELETE /orders/{orderNumber}

Returns {"deleted": 0} or {"deleted": 1}.

```
In [15]: resp = requests.delete(f"{BASE_URL}/orders/{new_order_id}", timeout=30)
assert resp.status_code == 200
assert resp.json()["deleted"] == 1

gone = requests.get(f"{BASE_URL}/orders/{new_order_id}", timeout=30)
assert gone.status_code == 404
gone.json()
```

Out [15]: {'detail': 'No order with orderNumber '99999'')}

Order Details

GET /orderdetails

List/search with optional query parameters: orderNumber , productCode (equality template).

```
In [16]: resp = requests.get(f"{BASE_URL}/orderdetails", timeout=30)
assert resp.status_code == 200, resp.text
data = resp.json()
assert "items" in data
print("count:", len(data["items"]))
data["items"][:3]
```

count: 2996

```
Out[16]: [{ 'orderNumber': 10100,
  'productCode': 'S18_1749',
  'quantityOrdered': 30,
  'priceEach': '136.00',
  'orderLineNumber': 3},
{ 'orderNumber': 10100,
  'productCode': 'S18_2248',
  'quantityOrdered': 50,
  'priceEach': '55.09',
  'orderLineNumber': 2},
{ 'orderNumber': 10100,
  'productCode': 'S18_4409',
  'quantityOrdered': 22,
  'priceEach': '75.46',
  'orderLineNumber': 4}]
```

```
In [17]: resp = requests.get(
    f"{BASE_URL}/orderdetails", params={"orderNumber": SAMPLE_ORDER_NUMBER}, timeout=30
)
assert resp.status_code == 200
items = resp.json()["items"]
assert all(r["orderNumber"] == SAMPLE_ORDER_NUMBER for r in items)
print("line items for order", SAMPLE_ORDER_NUMBER, ":", len(items))
items
```

line items for order 10100 : 4

```
Out[17]: [{ 'orderNumber': 10100,
  'productCode': 'S18_1749',
  'quantityOrdered': 30,
  'priceEach': '136.00',
  'orderLineNumber': 3},
{ 'orderNumber': 10100,
  'productCode': 'S18_2248',
  'quantityOrdered': 50,
  'priceEach': '55.09',
  'orderLineNumber': 2},
{ 'orderNumber': 10100,
  'productCode': 'S18_4409',
  'quantityOrdered': 22,
  'priceEach': '75.46',
  'orderLineNumber': 4},
{ 'orderNumber': 10100,
  'productCode': 'S24_3969',
  'quantityOrdered': 49,
  'priceEach': '35.29',
  'orderLineNumber': 1}]
```

GET /orders/{orderNumber}/orderdetails

Returns all line items for an order.

```
In [18]: resp = requests.get(f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}/orderdetails", timeout=30)
assert resp.status_code == 200
data = resp.json()
assert "items" in data
data["items"]
```

```
Out[18]: [{'orderNumber': 10100,
  'productCode': 'S18_1749',
  'quantityOrdered': 30,
  'priceEach': '136.00',
  'orderLineNumber': 3},
 {'orderNumber': 10100,
  'productCode': 'S18_2248',
  'quantityOrdered': 50,
  'priceEach': '55.09',
  'orderLineNumber': 2},
 {'orderNumber': 10100,
  'productCode': 'S18_4409',
  'quantityOrdered': 22,
  'priceEach': '75.46',
  'orderLineNumber': 4},
 {'orderNumber': 10100,
  'productCode': 'S24_3969',
  'quantityOrdered': 49,
  'priceEach': '35.29',
  'orderLineNumber': 1}]
```

GET /orders/{orderNumber}/orderdetails/{productCode}

Returns a single line item by composite key. Returns **404** if not found.

```
In [19]: resp = requests.get(
    f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}/orderdetails/{SAMPLE_PRODUCT_CODE}",
    timeout=30,
)
assert resp.status_code == 200
resp.json()
```

```
Out[19]: {'orderNumber': 10100,
  'productCode': 'S18_1749',
  'quantityOrdered': 30,
  'priceEach': '136.00',
  'orderLineNumber': 3}
```

```
In [20]: missing = requests.get(
    f"{BASE_URL}/orders/{SAMPLE_ORDER_NUMBER}/orderdetails/NOTEXIST", timeout=30
)
assert missing.status_code == 404
missing.json()
```

```
Out[20]: {'detail': "No order detail with orderNumber '10100' and productCode 'NOTEXIST'"}
```

POST /orderdetails

Creates a line item. First creates a scratch order to satisfy the foreign key constraint, then cleans up both at the end.

```
In [21]: # Create a scratch order to attach the test detail to.
scratch_order = {
    "orderNumber": 99998,
    "orderDate": "2024-02-01",
    "requiredDate": "2024-02-15",
    "status": "In Process",
    "customerNumber": SAMPLE_CUSTOMER_NUMBER,
}
r = requests.post(f"{BASE_URL}/orders", json=scratch_order, timeout=30)
assert r.status_code == 200, r.text
test_order_number = int(r.json())

detail_payload = {
    "orderNumber": test_order_number,
    "productCode": SAMPLE_PRODUCT_CODE,
    "quantityOrdered": 5,
    "priceEach": "95.70",
    "orderLineNumber": 1,
}
resp = requests.post(f"{BASE_URL}/orderdetails", json=detail_payload, timeout=30)
assert resp.status_code == 200, resp.text
print("created detail for order:", resp.json())
resp.json()
```

created detail for order: 99998

Out[21]: '99998'

PUT /orders/{orderNumber}/orderdetails/{productCode}

Updates a single line item by composite key.

```
In [22]: # Uses `test_order_number` from the POST cell above – run that cell first.
detail_update = {
    "orderNumber": test_order_number,
    "productCode": SAMPLE_PRODUCT_CODE,
    "quantityOrdered": 10,
    "priceEach": "90.00",
    "orderLineNumber": 1,
}
resp = requests.put(
    f"{BASE_URL}/orders/{test_order_number}/orderdetails/{SAMPLE_PRODUCT_CODE}",
    json=detail_update,
    timeout=30,
)
assert resp.status_code == 200, resp.text
assert resp.json() == {"updated": 1}
requests.get(
    f"{BASE_URL}/orders/{test_order_number}/orderdetails/{SAMPLE_PRODUCT_CODE}",
    timeout=30,
).json()
```

Out[22]: {'orderNumber': 99998,
'productCode': 'S18_1749',
'quantityOrdered': 10,
'priceEach': '90.00',
'orderLineNumber': 1}

DELETE /orders/{orderNumber}/orderdetails/{productCode}

Returns {"deleted": 0} or {"deleted": 1}. Also cleans up the scratch order.

```
In [23]: resp = requests.delete(
    f"{BASE_URL}/orders/{test_order_number}/orderdetails/{SAMPLE_PRODUCT_CODE}",
    timeout=30,
)
assert resp.status_code == 200
assert resp.json()["deleted"] == 1

gone = requests.get(
    f"{BASE_URL}/orders/{test_order_number}/orderdetails/{SAMPLE_PRODUCT_CODE}",
    timeout=30,
)
assert gone.status_code == 404
print("detail gone:", gone.json())

# Clean up the scratch order.
r = requests.delete(f"{BASE_URL}/orders/{test_order_number}", timeout=30)
assert r.json()["deleted"] == 1
print("scratch order deleted")
```

```
detail gone: {'detail': "No order detail with orderNumber '99998' and productCode 'S18_1749'"}
scratch order deleted
```